

Detailed Notes on ACL (Access Control Lists) in Linux

In traditional Linux permissions, we have only **three permission sets**:

-  **User (Owner)**
-  **Group**
-  **Others**

This model works well for simple access control. But what if:

- You want to give **read access to one extra user**
- Or give **write access to multiple specific users**
- Or assign different permissions to multiple groups?

Traditional permissions **cannot handle this efficiently**.

👉 That's where **ACL (Access Control Lists)** come in.

ACL allows **more fine-grained permission control** beyond the standard **rwX** model.

Owner | **Group** | **Others**

With normal permissions, you can assign access to:

- One user (owner)
- One group
- Everyone else

But what if:

- You want to give access to **multiple specific users**?
- Or give a user special permission without changing group ownership?

👉 That is where **ACL** is used.

1 Why ACL is Needed

✗ Limitation of Traditional Permissions

Example:

```
-rw-r----- 1 ashvi devops file.txt
```

This means:

- Owner → read/write
- Group → read
- Others → no access

But what if:

- You want to give **read** permission to user **john** only?

You must either:

- Change group ownership (affects others)
- Or give permission to "others" (not secure)

Or give permission to others (unsafe)

⚠ Not flexible.

✓ Solution: ACL

ACL allows:

- Add permission for specific user without changing owner/group
 - Add permission for multiple groups
 - Set default permissions for directories
-

2 What is ACL?

An **Access Control List (ACL)** is a mechanism that:

- Provides **additional permissions**
- Allows assigning permissions to:
 - Multiple users
 - Multiple groups
- Extends standard Linux file permissions

ACL is supported by most Linux file systems like:

- ext4
- XFS
- Btrfs

ACL works on:

- Files
 - Directories
-

There are only **two types of ACL in Linux**:

- 1 Access ACL
 - 2 Default ACL
-

3 Types of ACL

1 Access ACL (Normal ACL)

What is it?

Access ACL controls **current permissions on a file or directory**.

It works just like normal permissions — but gives extra flexibility.

Applies To:

- Files 
- Directories 

♦ Example

You have a file:

```
-rw-r----- 1 ashvi devops file.txt
```

Now you want to give **read permission to user john**, without changing owner/group.

You run:

```
setfacl -m u:john:r file.txt
```

Now check:

```
getfacl file.txt
```

You will see:

```
user::rw-
```

user:john:r--

group::r--

mask::r--

other::---

👉 This is **Access ACL**
It directly affects **file.txt** right now.

♦ Important Point

Access ACL:

- Does NOT affect new files
 - Only applies to that specific file or directory
-

2 Default ACL (Directory Only)

📌 What is it?

Default ACL is used to set **automatic permissions for new files created inside a directory**.

It works like a **template**.

📌 Applies To:

- Directories ONLY !
 - Not individual files
-

♦ Example

You have a directory:

`mkdir project`

Now you want:

- 👉 Whenever someone creates a new file inside **project**,
- 👉 User john should automatically get read/write permission.

You set:

`setfacl -m d:u:john:rw project`

Now check:

`getfacl project`

You will see:

`default:user:john:rw-`

Now create a file inside it:

`touch project/test.txt`

Check permissions:

`getfacl project/test.txt`

You will see john already has permission.

- 👉 This happened because of **Default ACL**.

Quick Comparison Table

Feature	Access ACL	Default ACL
Applies to	File & Directory	Directory only
Affects existing file	✅ Yes	❌ No

Affects new files	 No	 Yes
Used with <code>d :</code>	 No	 Yes
Example	<code>u:john:r</code>	<code>d:u:john:r</code>

4 Check if ACL is Enabled

Method 1: Check Mount Options

Run:

```
mount | grep xfs
```

If you see something like:

```
/dev/mapper/cs-root on / type xfs (rw,relatime,attr2,inode64,noquota)
```

Even if you don't see `acl` written explicitly —

 **ACL is still enabled in XFS.**

Because in XFS:

- ACL support is built-in
 - It does not always show `acl` in mount options
-

Method 2 (Best & Practical Way) – Real Test

This is the easiest and 100% confirmation method.

Step 1: Create a test file

```
touch test_acl
```

Step 2: Apply ACL

```
setfacl -m u:username:r test_acl
```

(Replace `username` with an existing user)

Step 3: Check ACL

```
getfacl test_acl
```

If ACL Is Enabled

You will see output like:

```
user::rw-  
user:username:r--  
group::r--  
mask::r--  
other::r--
```

AND

```
ls -l test_acl
```

Will show:

```
-rw-r--r--+ 1 root root 0 Mar 3 test_acl
```

👉 Notice the `+` sign

That means ACL is active.

If ACL Is NOT Enabled

You would get error like:

```
setfacl: test_acl: Operation not supported
```

But in your case (XFS), this will NOT happen.

Why You Don't Need to Edit /etc/fstab in Your Case

Because:

Filesystem	ACL Default Behavior
ext3	Must enable manually
ext4	Usually enabled
xfs	Always enabled

Your system is using:

xfs

So ACL is already working.

Enable ACL (If Not Enabled)

◆ Method 1: Temporary Enable (Until Reboot)

```
mount -o remount,acl /
```

This enables ACL immediately.

⚠ But after reboot, it may disappear.

◆ Method 2: Permanent Enable (Recommended)

Edit `/etc/fstab`:

```
vi /etc/fstab
```

Find your partition entry:

Example:

```
/dev/sda1 / ext4 defaults 0 1
```

Change it to:

```
/dev/sda1 / ext4 defaults,acl 0 1
```

Save and exit.

Now remount:

```
mount -o remount /
```

Or reboot the system.

5 ACL Commands

Command	Purpose
<code>getfacl</code>	View ACL
<code>setfacl</code>	Set ACL
<code>setfacl -x</code>	Remove ACL
<code>setfacl -b</code>	Remove all ACL

6 Viewing ACL

```
getfacl file.txt
```

Example output:

```
# file: file.txt
# owner: ashvi
# group: devops
user::rw-
user:john:r--
group::r--
mask::r--
other::---
```

7 Understanding ACL Entries

Entry	Meaning
user::	Owner permissions
user:john:	Specific user permission
group::	Group permission
mask::	Maximum effective permissions
other::	Others permission

8 Setting ACL for User

◆ Give read permission to john

```
setfacl -m u:john:r file.txt
```

Meaning:

- `-m` → modify
- `u` → user
- `john` → username
- `r` → read permission

Verify:

```
getfacl file.txt
```

◆ Give read & write

```
setfacl -m u:john:rw file.txt
```

9 Setting ACL for Group

```
setfacl -m g:devops:rw file.txt
```

10 Removing ACL

Remove specific user ACL:

```
setfacl -x u:john file.txt
```

Remove all ACL:

```
setfacl -b file.txt
```

11 Understanding Mask in ACL

In ACL, **mask** defines:

🎯 The maximum effective permission allowed for:

- Named users (`user:john`)
- Named groups (`group:dev`)
- Owning group (`group:.`)

Think of mask as:

🚧 A permission limiter / boundary

Even if you give full permission to a user, mask can reduce it.

Example to Understand Mask

Step 1: Create file

```
touch file1
```

Step 2: Give john full permission

```
setfacl -m u:john:rwx file1
```

Now check:

```
getfacl file1
```

You may see:

```
user::rw-
```

```
user:john:rwx
```

```
group::r--
```

```
mask::rwx
```

```
other::r--
```

Everything is fine.

Now Change Mask

```
setfacl -m m:r file1
```

Check again:

```
getfacl file1
```

Now you will see:

```
user::rw-
```

```
user:john:rwx    #effective:r--
```

```
group::r--      #effective:r--
```

```
mask::r--
```

other::r--

See this line?

#effective:r--

- 👉 Even though john has **rw**x,
 - 👉 Mask is **r--**,
 - 👉 So john's real permission becomes **r--**
-

12 Directory ACL

Give user access to directory

```
setfacl -m u:john:rwX project/
```

Now john can:

- Read
 - Write
 - Enter directory
-

13 Default ACL (For Directories)

Default ACL ensures:
New files inherit ACL automatically.

Set default ACL:

```
setfacl -d -m u:john:rwX project/
```

Now any file created inside project/ will give john rwX.

View:

```
getfacl project/
```

You will see:

```
default:user:john:rwX
```

Recursive ACL

Apply ACL to entire directory:

```
setfacl -R -m u:john:rwX project/
```

Remove recursively:

```
setfacl -R -b /project
```

Recursive default ACL:

```
setfacl -R -d -m u:john:rwX project/
```

How To Remove Recursive Default ACL

There are below methods depending on what you want.

◆ Method 1: Remove Only John's Default ACL Recursively

If you want to remove only john's default ACL:

```
setfacl -R -x d:u:john project/
```

Explanation:

- `-R` → recursive
 - `-x` → remove entry
 - `d:u:john` → remove default user john
-

◆ Method 2: Remove ALL Default ACL Recursively (Recommended)

If you want to remove all default ACL entries:

```
setfacl -R -k project/
```

Explanation:

- `-R` → recursive
- `-k` → remove default ACL

👉 `-k` removes only **default ACL**, not normal access ACL.

◆ Method 3: Remove Everything (Access + Default ACL)

If you want to completely clean ACL:

```
setfacl -R -b project/
```

Explanation:

- `-b` → remove all ACL entries (access + default)

⚠ This removes everything.

🔍 How To Verify After Removal

Run:

```
getfacl project/
```

If default ACL removed, you will NOT see:

```
default:user:john:...
```

15 How to Identify File Has ACL

Run:

```
ls -l
```

If you see:

```
-rw-rw-r--+
```

The + sign means:

👉 ACL is applied.

16 Backup and Restore ACL

Backup ACL

```
getfacl -R /project > acl_backup.txt
```

Restore ACL

```
setfacl --restore=acl_backup.txt
```

17 Difference: ACL vs Traditional Permission

Feature	chmod	ACL
Basic permissions	✓	✓
Multiple users	✗	✓
Multiple groups	✗	✓
Fine-grained control	✗	✓
Default inheritance	✗	✓

18 When to Use ACL

Use ACL when:

- Multiple teams share directories
 - Project-based access control required
 - DevOps environments
 - Enterprise file servers
 - Application shared storage
-

19 Advantages of ACL

- ✓ Fine-grained access control
 - ✓ Better security management
 - ✓ Ideal for shared environments
 - ✓ Useful in enterprise servers
 - ✓ Prevents unnecessary permission exposure
-

20 Disadvantages of ACL

- ✗ Slightly complex
 - ✗ Harder to manage manually
 - ✗ Backup tools must support ACL
 - ✗ Not always enabled by default
-